



```
# regular expression
# r'' regular expression

import re
text = "Call me at 555-123-4567 or email piyush@gmail.com"

#find the email

phone = re.findall(r'\d{3}-\d{3}-\d{4}', text)
print(phone)

#now finding the email
email = re.findall(r'\w+@\w+\.\w+', text)
print(email)

#again removing all the digits
clean = re.sub(r'\d', '', text)
print(clean)

# re.sub(pattern, replacement, text)
```

```
['555-123-4567']
['piyush@gmail.com']
Call me at -- or email piyush@gmail.com
```

```
#stemming
```

```
from nltk.stem import PorterStemmer
stemmer = PorterStemmer()

words = ['running', 'runs', 'ran', 'studies', 'studying', 'beautiful']

for w in words:
    print(w, "->" , stemmer.stem(w))
```

```
running -> run
runs -> run
ran -> ran
studies -> studi
studying -> studi
beautiful -> beauti
```

```
#lemmatization
```

```
import spacy

nlp = spacy.load('en_core_web_sm')

doc = nlp("The children were running and the mice ate cheese.")
```

```
for token in doc:
    print(token.text, "->", token.lemma_)
```

```
The -> the
children -> child
were -> be
running -> run
and -> and
the -> the
mice -> mouse
ate -> eat
cheese -> cheese
. -> .
```

```
#stops words
```

```
import nltk
nltk.download('stopwords')
from nltk.corpus import stopwords

stop_words = set(stopwords.words('english'))

text = "The quick brown fox jumps over the lazy dog"

words = text.lower().split()
filtered = [w for w in words if w not in stop_words]

print(filtered)
```

```
['quick', 'brown', 'fox', 'jumps', 'lazy', 'dog']
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
```

1. How re.findall() Works Unlike re.sub() (which replaces text) or re.search() (which only finds the first match), re.findall() scans the entire string from left to right and extracts every single match it finds, returning them all as a list of strings.

If no matches are found, it returns an empty list [].

2. Breaking Down the Regex Pattern Piece-by-Piece The pattern r'\w+@\w+.\w+' acts as a literal blueprint for what an email address looks like. Here is what each component matches:

\w+: Matches the username.

\w stands for any "word character" (letters, digits, or underscores).

The + means "one or more" of these characters.

@: Matches the literal @ symbol in the email.

\w+: Matches the domain name (like gmail or yahoo).

Again, this looks for one or more letters, digits, or underscores.

.: Matches a literal period/dot (.).

Note: In regular expressions, a plain dot . means "match any character." To tell Python you want a literal period, you must "escape" it with a backslash .

\w+: Matches the top-level domain extension (like com, org, or edu).

3. Step-by-Step Execution on Your Text Given your specific variable:

Python text = "Call me at 555-123-4567 or email piyush@gmail.com" The Scan Begins: re.findall() scans through "Call me at 555-123-4567 or email ". None of this matches the pattern because there is no @ symbol or surrounding word characters.

The Match is Found: It hits piyush@gmail.com.

piyush matches the first \w+

@ matches @

gmail matches the second \w+

. matches .

com matches the final \w+

The Output: It grabs that entire match, puts it in a list, and continues scanning until the end of the string.

Because it found exactly one match, print(email) outputs:

